JSS Academy of Technical Education, Bengaluru-60
Dept. of Information Science and Engineering

Lab Manual

Big Data Analytics Lab [BIS701]
7th sem ISE

BDA Lab Programs

PRACTICAL COMPONENT OF IPCC

Sl.NO	Experiments (Java/Python/R)
1	Install Hadoop and Implement the following file management tasks in Hadoop: Adding files and directories Retrieving files Deleting files and directories. **Hint:** A typical Hadoop workflow creates data files (such as log files) elsewhere and copies them into HDFS using one of the above command line utilities.
2	Develop a MapReduce program to implement Matrix Multiplication
3	Develop a Map Reduce program that mines weather data and displays appropriate messages indicating the weather conditions of the day.
4	Develop a MapReduce program to find the tags associated with each movie by analyzing movie lens data.
5	Implement Functions: Count – Sort – Limit – Skip – Aggregate using MongoDB
6	Write Pig Latin scripts to sort, group, join, project, and filter the data.
7	Use Hive to create, alter, and drop databases, tables, views, functions, and indexes.
8	Implement a word count program in Hadoop and Spark.
9	Use CDH (Cloudera Distribution for Hadoop) and HUE (Hadoop User Interface) to analyze data and generate reports for sample datasets

BDA Lab Programs:

Hadoop installation and HDFS commands

Program 1: Install Hadoop and Implement the following file management tasks in Hadoop:

Hadoop is an open-source framework used to **store and process Big Data**.
Main components:

- **HDFS** – stores large files across a distributed system
- **MapReduce** – processes data in parallel
- **YARN** – manages cluster resources
- **Hadoop Common** – common libraries and utilities

For Hadoop lab, Since we are using the Cloudera QuickStart VM, the procedure is:

1. Install VirtualBox on Windows.
2. Import/start the Cloudera QuickStart VM.
3. Start the Cloudera VM.
4. Open the Linux terminal.
5. Start/check Hadoop services.
6. Create files in HDFS.
7. Run Python MapReduce programs.
8. View the output using the terminal or Hadoop/HDFS web interface.

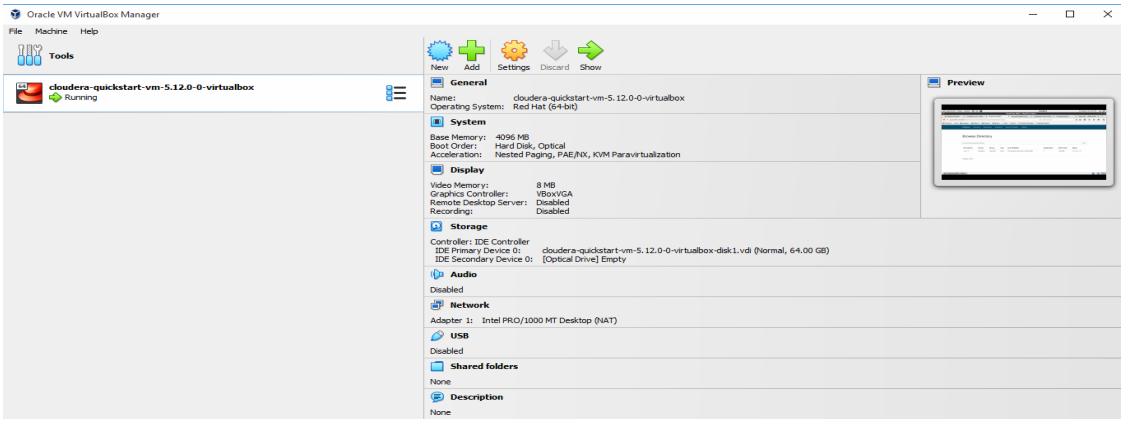
Cloudera: Cloudera QuickStart VM is a preconfigured virtual machine containing Linux and Hadoop-related software. It is useful for learning Hadoop without manually installing every component.
Cloudera provides a Hadoop-based platform/distribution that packages Hadoop and related Big Data tools, making them easier to install and use.
In the Cloudera QuickStart environment commonly used for labs, you find tools such as: Hadoop ,HDFS, MapReduce, YARN , Hive , HBase , Sqoop , Flume , Hue

Hadoop	Cloudera
Big Data framework	Platform/distribution built around Big Data tools
Provides HDFS and MapReduce	Packages Hadoop with additional tools and management interfaces
Core technology	Easier environment for installation, learning, and administration

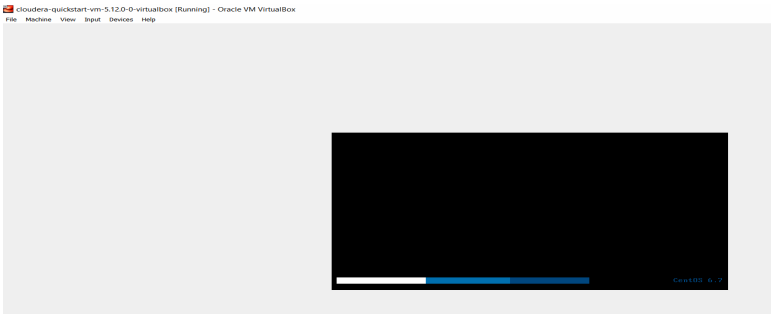
check Appendix-A, for hadoop installation on Cloudera VM
Cloudera QuickStart VM — Step-by-Step Procedure for doing **BDA/Hadoop lab with Cloudera QuickStart VM**, use the following procedure
Windows → VirtualBox → Cloudera → Hadoop → HDFS → MapReduce.

> **note:** Cloudera QuickStart VM is an old, preconfigured CDH environment intended for learning. It already contains Hadoop and related components, so you normally **do not install Hadoop separately inside the VM**. Cloudera's archived documentation confirms that the QuickStart VM includes CDH and Hadoop tools.

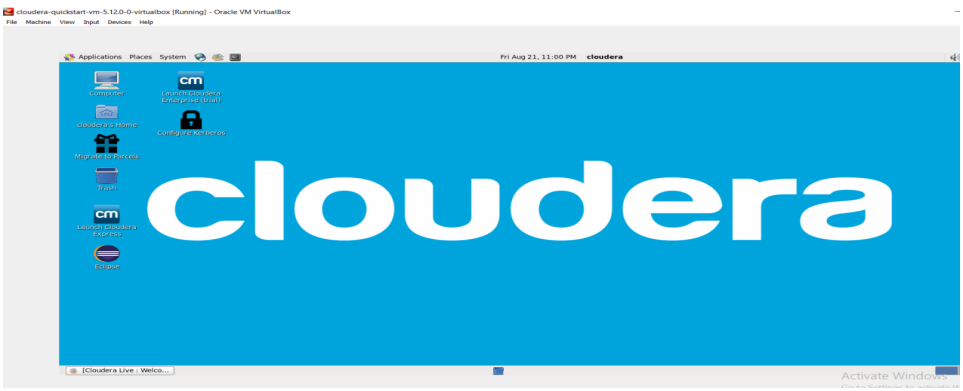
SNAPSHOTS: hadoop environment
Virtual Box



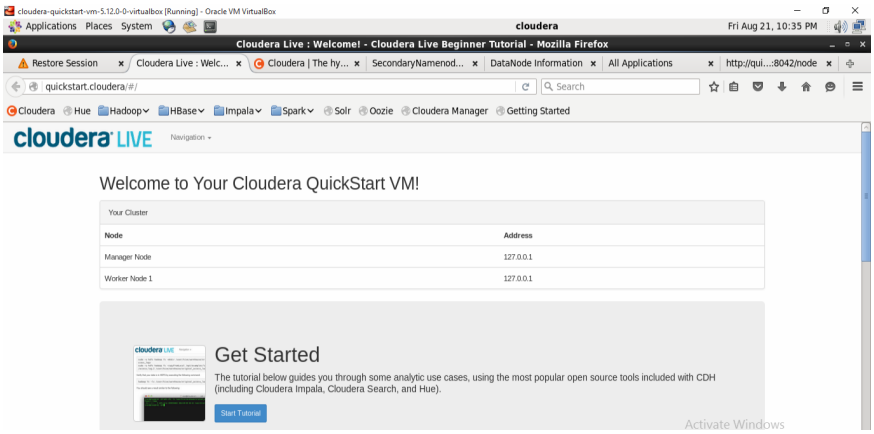
loading.....Cent OS



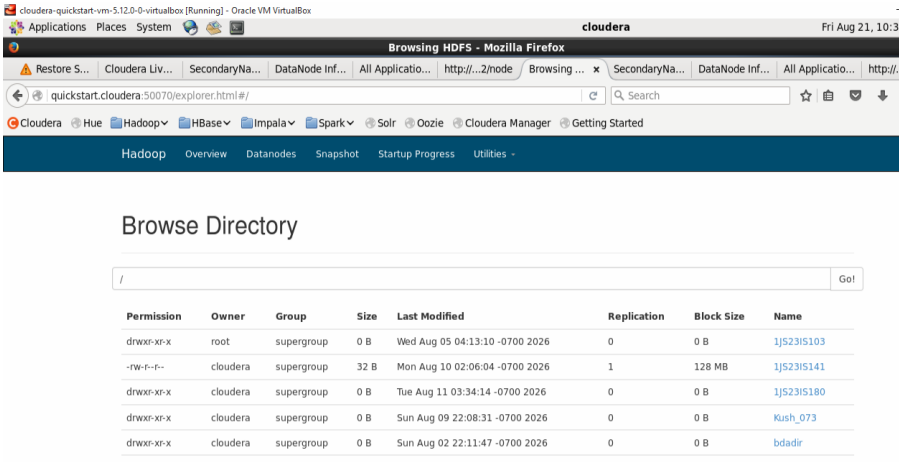
Cloudera: Desktop



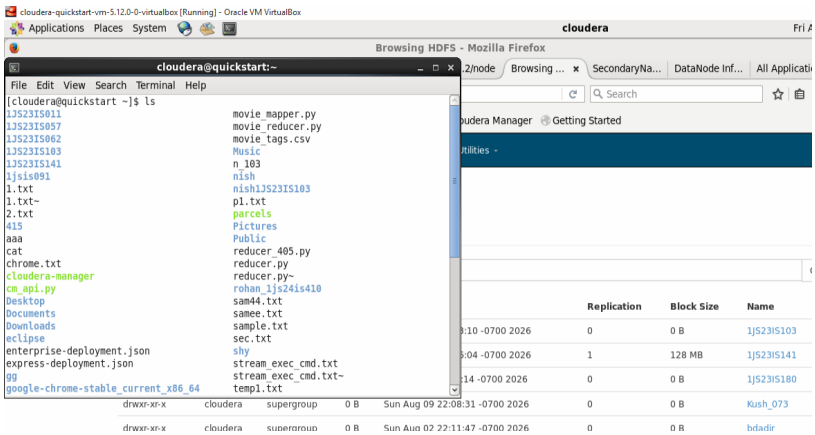
Cloudera: Hadoop frame-work



Hadoop - Data storage, HDFS - File system



Local User Terminal - command prompt, File system



HDFS commands

Program 1: Install Hadoop and Implement the following file management tasks in Hadoop:

- Adding files and directories;
- Retrieving files;
- Deleting files and directories.

Sl. No.	Hadoop/ HDFS commands	Description
1.	hdfsdfs -ls / **Lists the files and directories** at the root (/) of the Hadoop Distributed File System (HDFS). Its similar to the Unix/Linux ls command but for HDFS.	
2.	hdfsdfs -ls -R / -R (uppercase R) means **recursive listing**. This will list all files and directories inside /, the root directory and all its subdirectories.	
	Hdfs commands ,for Adding files and directories	
3.	hdfsdfs -mkdir /bdadir Creates a new directory named **bdadir** at the root (/) of the HDFS file system. Similar to mkdir in Linux but on HDFS.	
4.	hdfsdfs -mkdir /bdadir/pg1 Creates a directory called **pg1** inside the existing /bdadir directory. So now the path /bdadir/pg1 exists in HDFS.	
5.	echo hello world, its a bright day > sampl.txt Writes the text **hello world statement** into a file named sampl.txt. If sampl.txt **doesn't exist**, it will be created. If it **exists**,it will **overwrite** with the new text.	
	Hdfs commands, for Retrieving files	
6.	hdfsdfs -put sampl.txt /bdadir/pg1/ Put uploads sampl.txt from your local file system into the /bdadir/pg1/ directory in HDFS.	
7.	hdfsdfs -copyFromLocal sampl.txt /bdadir/pg1/ **Copies the local file**sampl.txt from your machine's file system **into the HDFS directory**/bdadir/pg1/. Equivalent to hdfsdfs -put sampl.txt /bdadir/pg1/. Useful for uploading data files from local machine to HDFS.	
8.	hdfsdfs -get /bdadir/pg1/sampl.txt . Downloads the file /bdadir/pg1/sampl.txt from **HDFS** to the **local current directory** (.). -get command is used to copy files **from HDFS to your local file system**.	
9.	hdfsdfs -copyToLocal /bdadir/pg1/sampl.txt . Copies the file /bdadir/pg1/sampl.txt**from HDFS to your local machine's current directory**(.).Same effect as hdfsdfs -get — both commands work similarly for downloading files from HDFS.	
10.	hdfsdfs -cat /bdadir/pg1/sampl.txt	

	Displays the **contents of the file**/bdadir/pg1/sampl.txt directly in the terminal. Useful to quickly check what's inside an HDFS file without copying it locally.
	Hdfs commands for, Deleting files and directories.
11.	hdfsdfs -rm /bdadir/pg1/sampl.txt **Removes (deletes) the file**sampl.txt from the HDFS directory /bdadir/pg1/. Once deleted, the file is gone from HDFS (no recycle bin).
12.	Hdfsdfs -rmdir /bdadir/pg1 Removes the directory /bdadir/pg1**from HDFS**. **Only works if the directory is empty** (no files or subdirectories inside). If the directory contains files, it will give an error and **won't delete**.
13.	hdfsdfs -rm -r /bdadir/pg1 If the directory is **not empty**and want to delete it along with everything inside,
14.	Use -rmdir for **empty directories** Use -rm -r for **non-empty directories** (recursive delete).
15.	hdfsdfs -cp /bdadir/pg1/sampl.txt /bdadir/sampl22.txt **Copies** the file sampl.txt from /bdadir/pg1/**to**/bdadir/ with the new name sampl22.txt. Effectively creates a duplicate of the file in a different location or with a different name within HDFS.
16.	hdfs dfs -mv /bdadir/pg1/sampl.txt /bdadir/sampl22.txt **Moves** (or renames) the file sampl.txt from /bdadir/pg1/ to /bdadir/ and renames it to sampl22.txt.The original file is removed from /bdadir/pg1/ and appears at /bdadir/sampl22.txt. It's like a **cut-and-paste** operation inside HDFS.

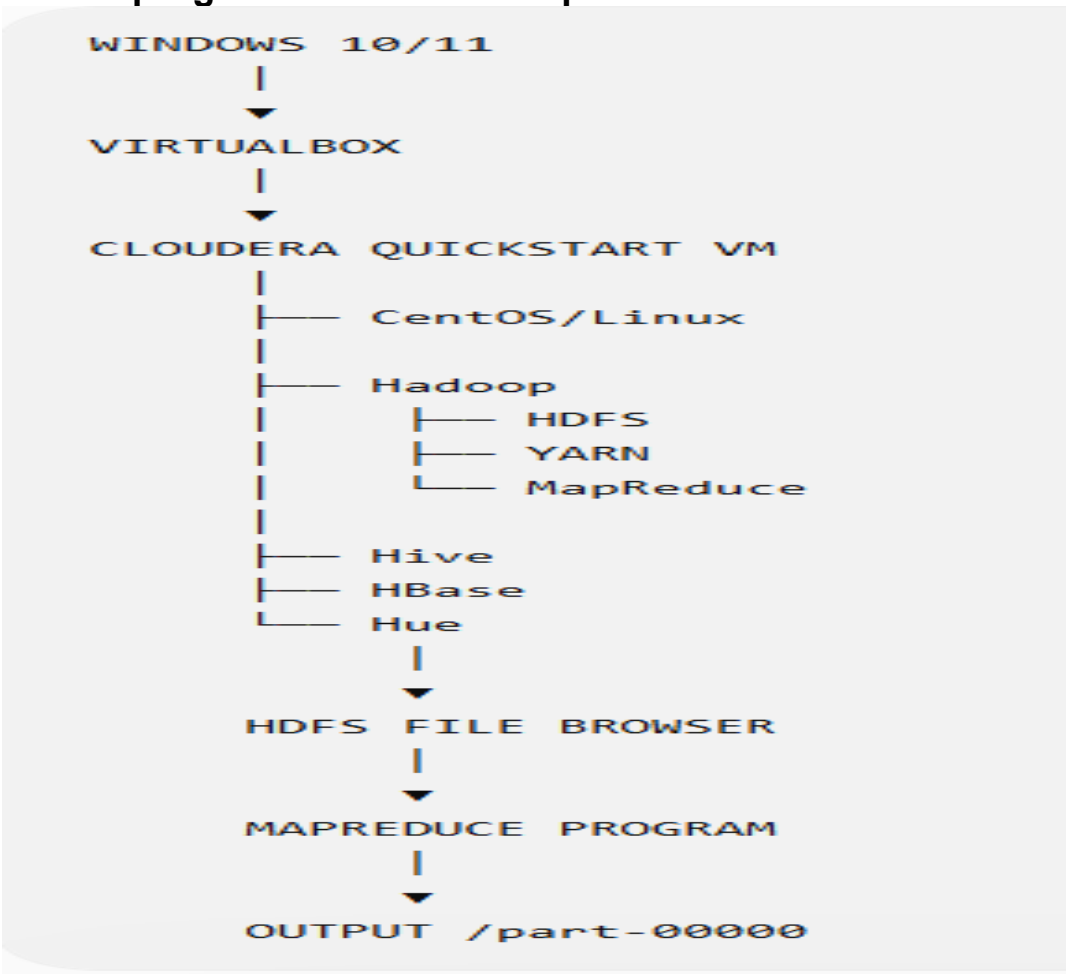
for more Hadoop Commands, refer, Appendix-B.

Map-Reduce programming

HDFS Commands for creating and executing, Map-Reduce programs in Python

- **Create and check the files in Linux folder**
  `gedit filename.py , ls , ls -l`
- **Make Python programs executable**
  `chmod +x wc_mapper.py`
  `chmod +x wc_reducer.py`

Map-Reduce program execution Complete flow



- **Execute Map-Reduce Python programs by two compilation methods**

Method 1: **Compilation command**

```
cat text.txt | python wc_mapper.py

cat text.txt | python wc_mapper.py | sort | python wc_reducer.py
```

Method 2 : Hadoop streaming command (follow 4 steps)

```
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -input /usr/cloudera/wordcount/input -output /usr/cloudera/wordcount/output -mapper "python2 wc_mapper.py" -reducer "python2 wc_reducer.py" -file wc_mapper.py -file wc_reducer.py
```

Steps for Method 2 — To Run a MapReduce job as Hadoop Streaming at Cloudera Hadoop/HDFS.

1] put the input file into HDFS

Command to create Input directory and to put input file and check it ***

```
hdfs dfs -mkdir -p /usr/cloudera/wordcount/input

hdfs dfs -put wordcnt.txt /usr/cloudera/wordcount/input/

hdfs dfs -ls /usr/cloudera/wordcount/input
```

2] remove the output file at HDFS

Command to delete output directory (always do it, before next execution)

```
hdfs dfs -rm -r /usr/cloudera/wordcount/output
```

3] run the hadoop streaming cmd with arguments

****Command for Hadoop streaming ****

```
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -input /usr/cloudera/wordcount/input -output /usr/cloudera/wordcount/output -mapper "python2 wc_mapper.py" -reducer "python2 wc_reducer.py" -file wc_mapper.py -file wc_reducer.py
```

refer **Appendix - C** , for the description of the Hadoop Streaming command

Complete meaning of Hadoop streaming command:

*Use Hadoop Streaming to read weather data from HDFS, execute **wc_mapper.py** using Python 2, shuffle and sort the Mapper output, execute **wc_reducer.py** using Python 2, and store the final result in the HDFS output directory.*

4] check the output file at HDFS

*** Command to check the output *****

```
hdfs dfs -ls /usr/cloudera/wordcount/output

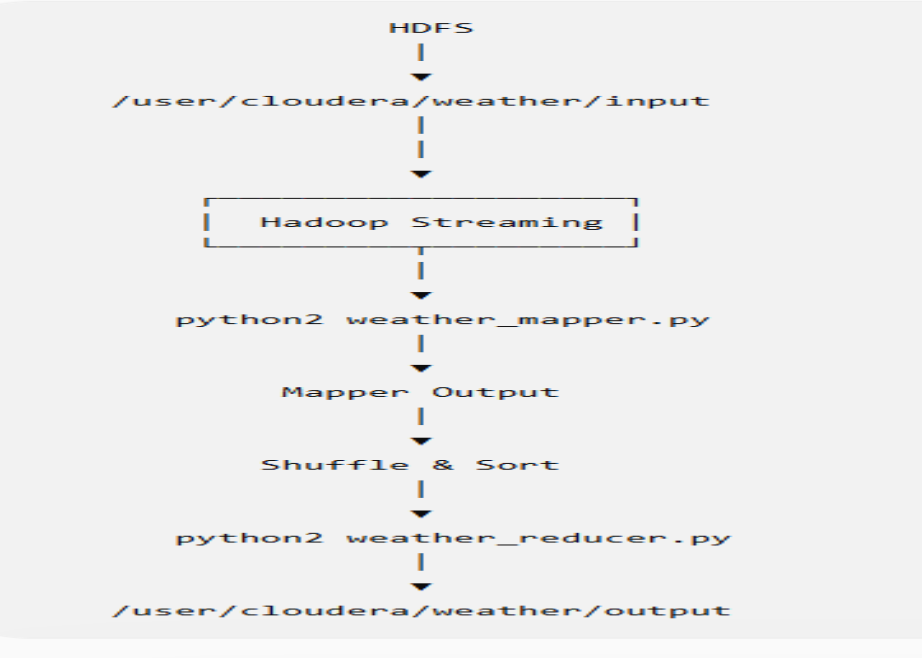
hdfs dfs -cat /usr/cloudera/wordcount/output/part-00000
```

What happens when we press Enter? ie. when streaming command is executed

Hadoop goes through these stages:
Stage 1 — Read input, Hadoop reads: /usr/cloudera/weather/input
from HDFS.
Stage 2 — Start Mapper, It executes: python2 weather_mapper.py

Stage 3 — Mapper processes data, The Mapper reads each input record and generates intermediate key-value pairs.
Stage 4 — Shuffle and Sort, Hadoop automatically groups identical keys together.
Stage 5 — Start Reducer, It executes: python2 weather_reducer.py
Stage 6 — Write result
The final output is written to: /usr/cloudera/weather/output

Usually you will find two o/p files: _SUCCESS, part-00000



To Check Output: View HDFS through the Browser

- To see the HDFS files in the browser.
- Inside the Cloudera VM, open the browser and use the **Hue** interface.
- The QuickStart VM provides Hue as a web UI for interacting with Hadoop.
- use the **File Browser** to navigate to:

/user/cloudera/weather --> /user/cloudera/weather/input
or
/user/cloudera/weather/output
For your MapReduce result, open: /user/cloudera/weather/output/part-00000

Map-Reduce programming

2. Develop a Map-Reduce program to implement Matrix Multiplication

Multiply, two **2 × 2 matrices**:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Matrix Multiplication program, mat_mapper.py

mat_mapper.py

```
#!usr/bin/env python
import sys

A_ROWS = 2
B_COLS = 2

for line in sys.stdin:
        line = line.strip()
        if not line:
                continue
        parts = line.split()
        if len(parts) != 4:
                continue
        matrix, row, col, value = parts
        row = int(row)
        col = int(col)
        value = int(value)

        if matrix == "A":
                for k in range(B_COLS):
                        sys.stdout.write("%d %d\tA %d %d\n" % (row, k, col, value))
        elif matrix == "B":
                for i in range(A_ROWS):
                        sys.stdout.write("%d %d\tB %d %d\n" % (i, col, row, value))
```

****

Matrix Multiplication program, mat_reducer.py

mat_reducer.py

```
import sys
from collections import defaultdict

intermediate = defaultdict(list)

for line in sys.stdin:
        line = line.strip()
        if not line:
                continue
        key, value = line.split("\t")
        i_str, j_str = key.split()
        i = int(i_str)
        j = int(j_str)
        parts = value.split()

        if len(parts) != 3:
                continue
        m = parts[0]
        k = int(parts[1])
        v = int(parts[2])
        intermediate[(i,j)].append((m,k,v))

for(i,j), values in intermediate.items():
        a_values = dict((k,v) for (m,k,v) in values if m == "A" )
        b_values = dict((k,v) for (m,k,v) in values if m == "B" )
        result = 0

        for k, av in a_values.items():
                if k in b_values:
                        result += av * b_values[k]
        if result != 0:
                sys.stdout.write("%d %d %d \n" % (i,j,result))
```

Matrix Multiplication program, mat_reducer.py

```
import sys
from collections import defaultdict
intermediate = defaultdict(list)

for line in sys.stdin:
        line = line.strip()
        if not line:
                continue
        key, value = line.split("\t")
        i_str, j_str = key.split()
        i = int(i_str)
        j = int(j_str)
        parts = value.split()

        if len(parts) != 3:
                continue
        m = parts[0]
        k = int(parts[1])
```

```
        v = int(parts[2])
        intermediate[(i,j)].append((m,k,v))

    for(i,j), values in intermediate.items():
        a_values = dict((k,v) for (m,k,v) in values if m == "A" )
        b_values = dict((k,v) for (m,k,v) in values if m == "B" )
        result = 0

        for k, av in a_values.items():
            if k in b_values:
                result += av * b_values[k]
        if result != 0:
            sys.stdout.write("%d %d %d \n" % (i,j,result))
```

****

Matrix Multiplication program, mat_mapper.py

```
#!usr/bin/env python
import sys
A_ROWS = 2
B_COLS = 2
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    parts = line.split()
    if len(parts) != 4:
        continue
    matrix, row, col, value = parts
    row = int(row)
    col = int(col)
    value = int(value)

    if matrix == "A":
        for k in range(B_COLS):
            sys.stdout.write("%d %d\tA %d %d\n" % (row, k, col,
value))
    elif matrix == "B":
        for i in range(A_ROWS):
            sys.stdout.write("%d %d\tB %d %d\n" % (i, col, row,
value))
```

****

INPUT File: matrix.txt

A 0 0 3
A 0 1 2
A 1 0 1
A 1 1 4

B 0 0 2
B 1 0 5
B 0 1 3
B 1 1 1

OUTPUT: Matrix Multiplication

[cloudera@quickstart ~]$ cat matrix.txt | python mat_mapper.py

0 0 A 0 3
0 1 A 0 3
0 0 A 1 2
0 1 A 1 2
1 0 A 0 1
1 1 A 0 1
1 0 A 1 4
1 1 A 1 4
0 0 B 0 2
1 0 B 0 2
0 0 B 1 5
1 0 B 1 5
0 1 B 0 3
1 1 B 0 3
0 1 B 1 1
1 1 B 1 1

[cloudera@quickstart ~]$ cat matrix.txt | python mat_mapper.py | sort | python mat_reducer.py

0 1 11
1 0 22
0 0 16
1 1 7

****

Resultant matrix with location =

16	**11**
(00)	**(01)**
22	**7**
(10)	**(11)**

3. Develop a Map Reduce program that mines weather data and displays appropriate messages indicating the weather conditions of the day

Weather data Analysis, weather_mapper.py

weather_mapper.py

```
#!/usr/bin/env python
import sys

for line in sys.stdin:
    line = line.strip()
    parts = line.split(',')
    if len(parts) == 3:
        date = parts[0]
        temp = parts[1]
        condition = parts[2]
        print '%s\t%s\t%s' % (date,temp,condition)
```

Weather data Analysis, weather_reducer.py

weather_reducer.py

```
#!/usr/bin/env python
import sys

for line in sys.stdin:
    line = line.strip()
    date, temp, condition = line.split('\t')

    try:
        temp = int(temp)
    except ValueError:
        continue

    if temp > 35:
        message = "Hot Day"
    elif temp >= 25 and temp <= 35:
        message = "Pleasant Day"  # Fixed typo
    else:
        message = "Cold Day"
    print '%s\t%s\t%s\t%s' % (date, temp, condition, message)
```

****

Weather data Analysis, weather_mapper.py

```
#!/usr/bin/env python
import sys
for line in sys.stdin:
   line = line.strip()
   parts = line.split(',')
   if len(parts) == 3:
        date = parts[0]
      temp = parts[1]
      condition = parts[2]
      print '%s\t%s\t%s' % (date,temp,condition)
```

****

Weather data Analysis, weather_reducer.py

```
#!/usr/bin/env python
import sys
for line in sys.stdin:
   line = line.strip()
   date, temp, condition = line.split('\t')
   try:
      temp = int(temp)
   except ValueError:
      continue
   if temp > 35:
      message = "Hot Day"
   elif temp >= 25 and temp <= 35:
      message = "Pleasant Day"  # Fixed typo
   else:
      message = "Cold Day"
   print '%s\t%s\t%s\t%s' % (date, temp, condition, message)
```

****

INPUT File: weather.txt

2025-09-08,32,Sunny

2025-09-07,24,Rainy

2025-09-06,18,Cloudy

2025-09-05,40,Sunny

****

OUTPUT: Weather data Analysis

[cloudera@quickstart ~]$ cat weather.txt | python weather_mapper.py

2025-09-08 32 Sunny

2025-09-07 24 Rainy

2025-09-06 18 Cloudy

2025-09-05 40 Sunny

[cloudera@quickstart ~]$ cat weather.txt | python weather_mapper.py | sort | python weather_reducer.py

2025-09-05 40 Sunny Hot Day

2025-09-06 18 Cloudy Cold Day

2025-09-07 24 Rainy Cold Day

2025-09-08 32 Sunny Pleasant Day

****

4. Develop a MapReduce program to find the tags associated with each movie by analyzing movie lens data.

Movie data Analysis, movie_mapper.py

movie_mapper.py

```
#!/usr/bin/env python
import sys

for line in sys.stdin:
    line = line.strip()
    parts = line.split(',')

    if len(parts) >= 3 and parts[1] != "movieId":
        movieId = parts[1]
        tag = parts[2]
        print "%s\t%s" % (movieId, tag)
```

Movie data Analysis, movie_reducer.py

movie_reducer.py

```
#!/usr/bin/env python
import sys

current_movie = None
tags = []

for line in sys.stdin:
    line = line.strip()
    movieId, tag = line.split('\t')

    if current_movie == movieId:
        tags.append(tag)
    else:
        if current_movie:
            print "%s\t%s" % (current_movie, ",".join(set(tags)))
        current_movie = movieId
        tags = [tag]

if current_movie:
    print "%s\t%s" % (current_movie, ",".join(set(tags)))
```

****

Movie data Analysis, movie_mapper.py

```
#!/usr/bin/env python
import sys
for line in sys.stdin:
   line = line.strip()
   parts = line.split(',')
   if len(parts) >= 3 and parts[1] != "movieId":
      movieId = parts[1]
      tag = parts[2]
      print "%s\t%s" % (movieId, tag)
```

****

Movie data Analysis, movie_reducer.py

```
#!/usr/bin/env python
import sys
current_movie = None
tags = []
for line in sys.stdin:
   line = line.strip()
   movieId, tag = line.split('\t')
   if current_movie == movieId:
      tags.append(tag)
   else:
      if current_movie:
         print "%s\t%s" % (current_movie, ",".join(set(tags)))
      current_movie = movieId
      tags = [tag]
if current_movie:
   print "%s\t%s" % (current_movie, ",".join(set(tags)))
```

****

INPUT File: movie_tags.csv

userId,movieId,tag,timestamp

15,1,classic,1138537770

15,1,pixar,1138537770

23,2,thriller,1138537770

****

OUTPUT: Movie data Analysis

[cloudera@quickstart ~]$ cat movie_tags.csv | python movie_mapper.py

1 classic

1 pixar

2 thriller

[cloudera@quickstart ~]$ cat movie_tags.csv | python movie_mapper.py | sort | python movie_reducer.py

1 pixar,classic

2 thriller

****

8. Implement a word count program in Hadoop .

wc_mapper.py

```
import sys

for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
        print "%s\t1" % word
```

---

wc_reducer.py

```
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.strip().split("\t")
    count = int(count)

    if current_word == word:
        current_count += count
    else:
        if current_word is not None:
            print "%s\t%d" % (current_word, current_count)

        current_word = word
        current_count = count

if current_word is not None:
    print "%s\t%d" % (current_word, current_count)
```

word count, wc_mapper.py

```
import sys

for line in sys.stdin:

   line = line.strip()

   words = line.split()

   for word in words:

      print "%s\t1" % word
```

Word count, reducer2.py

```
import sys

current_word = None
```

```
current_count = 0
for line in sys.stdin:
   word, count = line.strip().split("\t")
   count = int(count)

   if current_word == word:
      current_count += count
   else:
      if current_word is not None:
         print "%s\t%d" % (current_word, current_count)

      current_word = word
      current_count = count

if current_word is not None:
   print "%s\t%d" % (current_word, current_count)
```

****

Input FILE: wordcnt.txt

hello world , JSS

hello JSS 7 'B'

hello get msg

****

Method 1: Compilation command

cat text.txt | python wc_mapper.py

cat text.txt | python wc_mapper.py | sort | python wc_reducer.py

Method 2 : Hadoop streaming command (follow 4steps)

hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -input /usr/cloudera/wordcnt/input -output /usr/cloudera/wordcnt/output -mapper "python2 wc_mapper.py" -reducer "python2 wc_reducer.py" -file wc_mapper.py -file wc_reducer.py

Steps for Method 2 — To Run a MapReduce job as Hadoop Streaming at Cloudera Hadoop/HDFS.

1] put the input file into HDFS

Command to create Input directory and to put input file and check it ***

hdfs dfs -mkdir -p /usr/cloudera/wordcount/input

hdfs dfs -put weather.txt /usr/cloudera/wordcount/input/

hdfs dfs -ls /usr/cloudera/wordcount/input

2] remove the output file at HDFS

Command to delete output directory (always do it, before next execution)

hdfs dfs -rm -r /usr/cloudera/wordcount/output

3] run the hadoop streaming cmd with arguments

****Command for Hadoop streaming ****

hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -input /usr/cloudera/wordcount/input -output /usr/cloudera/wordcount/output -mapper "python2 wc_mapper.py" -reducer "python2 wc_reducer.py" -file wc_mapper.py -file wc_reducer.py

4] check the output file at HDFS

*** Command to check the output *****

hdfs dfs -ls /usr/cloudera/weather/output

hdfs dfs -cat /usr/cloudera/weather/output/part-00000

appendix

appendix-a

Hadoop installation on Cloudera VM

Cloudera QuickStart VM — Step-by-Step Procedure for doing **BDA/Hadoop lab with Cloudera QuickStart VM**, use the following procedure
Windows → VirtualBox → Cloudera → Hadoop → HDFS → MapReduce.

> **note:** Cloudera QuickStart VM is an old, preconfigured CDH environment intended for learning. It already contains Hadoop and related components, so you normally **do not install Hadoop separately inside the VM**. Cloudera's archived documentation confirms that the QuickStart VM includes CDH and Hadoop tools.

PART 1 — Install VirtualBox on Windows
Step 1: Download VirtualBox
Open your Windows browser and go to the official VirtualBox website: VirtualBox Downloads
Download the **Windows hosts** version.
Step 2: Install VirtualBox

PART 2 — Download Cloudera QuickStart VM
For the type of environment we have been using, **Cloudera QuickStart VM 5.13.0 for VirtualBox** is appropriate. archived QuickStart documentation identifies VirtualBox as one of the supported VM formats.
The commonly used 5.13.0 VirtualBox archive is:
Cloudera QuickStart VM 5.13 VirtualBox download
Step 3: Extract the ZIP
cloudera-quickstart-vm-5.13.0-0-virtualbox.zip
After extraction, you should have the VirtualBox appliance files.

PART 3 — Import Cloudera into VirtualBox
Step 4: Open VirtualBox
From Windows: Start -> VirtualBox
Open **Oracle VM VirtualBox**.
Step 5: Import the Cloudera VM
In VirtualBox: File - > Import Appliance
A window appears. Click the **folder/browse icon**. Go to the folder where you extracted Cloudera.
Select the Cloudera appliance file, typically:
cloudera-quickstart-vm-5.13.0-0-virtualbox.ovf
Then: Open -> Next -> Import
The general import sequence is also documented in Cloudera QuickStart installation guides: extract the VM, open VirtualBox, use **File → Import Appliance**, select appliance, and import it.

PART 4 — Configure the Virtual Machine
After importing, you should see something similar to:
Cloudera-quickstart-vm-5.13.0-0
on the left side of VirtualBox.
Step 6: Select the VM
Click: **Cloudera-quickstart-vm-5.13.0-0**
Then click: **Settings**
Step 7: Check Memory
Go to: Settings → System → Motherboard
Set the VM memory according to your computer's available RAM.
For the QuickStart VM, Cloudera's archived documentation gives **4+ GiB** as the minimum recommended VM memory for the basic CDH 5 configuration, with more needed for heavier configurations.
If your computer has **8 GB RAM**, don't allocate all 8 GB to the VM.
A practical starting point is: Computer RAM: 8 GB VM RAM: 4 GB
If your computer has 16 GB: Computer RAM: 16 GB VM RAM: 6–8 GB
Step 8: Check CPU
Go to: Settings → System → Processor

Use approximately: 2 CPUs for normal lab work.

PART 5 — Start Cloudera
Step 9: Start the VM
Click: **Start**
You will see a black Linux boot screen.
Do not shut it down while Hadoop services are starting.
The QuickStart VM can take several minutes because multiple Hadoop-related services start during boot.
Eventually you should reach the Cloudera Linux desktop.

PART 6 — Login
If a login screen appears, use:
Username: cloudera
Password: cloudera
The QuickStart documentation identifies cloudera / cloudera as the default credentials for the environment.

PART 7 — Open the Cloudera/Hadoop Interface
After the desktop appears, you may see a browser already open.
The QuickStart VM is designed to provide a browser-based interface for Hadoop tools, including Hue.
If the browser does not open automatically, open **Firefox/Chrome inside the VM**.
For the Hadoop/Hue interface, you can typically use:
http://quickstart.cloudera:8888
or:
http://localhost:8888
The exact interface available can depend on which services have started.

PART 8 — Open the Terminal
For your BDA lab, this is the most important screen.
Right-click the desktop and select: Open Terminal
You should get something similar to:
[cloudera@quickstart ~] $
This is where you will enter your Hadoop commands.

PART 9 — Check Hadoop
Step 10: Check Hadoop version
Enter: hadoop version
You should see information containing a Hadoop/CDH version.
Your earlier lab output showed: hadoop-streaming-2.6.0-cdh5.12.0.jar
so your environment is a **CDH 5.x / Hadoop 2.6.x** environment.

PART 10 — Check HDFS
Enter: hdfs dfs -ls /
You should get HDFS directory information.
For example: Found ...
drwxr-xr-x - hdfs supergroup ...
If this command works, **HDFS is running**.
****

appendix-b

Common Hadoop Commands: using Cloudera QuickStart VM,

1. Check Hadoop version

```
hadoop version
```

2. Check HDFS status

```
hdfs dfsadmin -report
```

3. List files in the HDFS root directory

```
hdfs dfs -ls /
```

hdfs commands for - Adding files and directories:

4. Create a directory in HDFS

```
hdfs dfs -mkdir /user/cloudera
```

Create nested directories:

```
hdfs dfs -mkdir -p /user/cloudera/bda_lab/input
```

5. List files in an HDFS directory

```
hdfs dfs -ls /user/cloudera
```

6. Create a local text file

```
echo "Hello Hadoop Hello Big Data" > input.txt
```

Check its contents: cat input.txt

Hdfs commands, for Retrieving files

7. Upload a local file to HDFS

```
hdfs dfs -put input.txt /user/cloudera/bda_lab/input
```

8. View a file stored in HDFS

```
hdfs dfs -cat /user/cloudera/bda_lab/input/input.txt
```

9. Copy a file from HDFS to the local system

```
hdfs dfs -get /user/cloudera/bda_lab/input/input.txt
```

10. Copy an entire HDFS directory to the local system

```
hdfs dfs -get /user/cloudera/bda_lab/input /home/cloudera/
```

Hdfs commands for, Deleting files and directories

11. Remove a file from HDFS

```
hdfs dfs -rm /user/cloudera/bda_lab/input/input.txt
```

12. Remove an HDFS directory

```
hdfs dfs -rm -r /user/cloudera/bda_lab/output
```

13. Rename or move a file

```
hdfs dfs -mv /user/cloudera/bda_lab/input/input.txt
/user/cloudera/bda_lab/input/data.txt
```

14. Copy a file within HDFS

```
hdfs dfs -cp /user/cloudera/bda_lab/input/data.txt
/user/cloudera/bda_lab/input/copy.txt
```

15. Check HDFS disk usage

```
hdfs dfs -du /user/cloudera/bda_lab
```

For a readable summary:

```
hdfs dfs -du -h /user/cloudera/bda_lab
```

---

1. jps #command to see all hadoop daemons.

2. hdfs dfs #List all the hadoop file system shell commands

3. hdfs dfs -help # ask for help!

4. hadoop fsck / -files -blocks #will list blocks that make up each file in filesystem

5.hdfs dfs -df hdfs:/

Report the amount of space used and available on currently mounted filesystem.

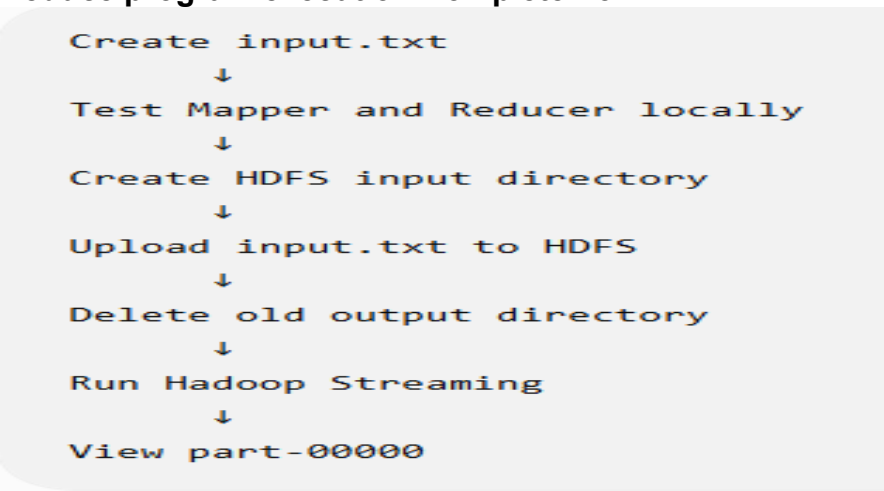
6. hdfs dfs -count /DRIVE

Count the number of directories,files and bytes under the paths that match the specified file pattern

appendix-c

Method 2 — to Run a Hadoop Streaming MapReduce job on Cloudera Hadoop/HDFS.

Python Map Reduce program execution Complete flow



Command to run a Python MapReduce program for weather data **using Hadoop Streaming in** Cloudera QuickStart VM**.**

```
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar
-input /usr/cloudera/weather/input
-output /usr/cloudera/weather/output
-mapper "python2 weather_mapper.py"
-reducer "python2 weather_reducer.py"
-file weather_mapper.py
-file weather_reducer.py
```

Execution Command description, part by part**.**

---

1. **hadoop**

`hadoop`

This invokes the Hadoop command-line program.

You are telling Cloudera: ″`I want Hadoop to execute a job.`″

---

2. **jar**

`hadoop jar`

`jar` tells Hadoop to execute a **Java JAR file**.

Hadoop Streaming is distributed as a JAR file.

---

3. Hadoop Streaming JAR

`/usr/lib/hadoop-mapreduce/hadoop-streaming.jar`

This is the location of the Hadoop Streaming JAR inside Cloudera VM.

Hadoop Streaming allows us to write Mapper and Reducer programs in languages such as: Python , Perl , Shell , Instead of writing Java MapReduce programs.

So our Python programs, can be executed by Hadoop.

`weather_mapper.py`

`weather_reducer.py`

4. `-input`

`-input /usr/cloudera/weather/input`

This tells Hadoop: "Read my input data from this HDFS directory."

The important point is that this is an **HDFS path**, not a normal Linux folder.

HDFS path structure is:

```
/user
   └── cloudera
        └── weather
             └── input
```

You can check it with:

```
hdfs dfs -ls /usr/cloudera/weather/input
```

You can see the input contents using:

```
hdfs dfs -cat /usr/cloudera/weather/input/*
```

5. `-output`

`-output /usr/cloudera/weather/output`

This tells Hadoop: "Put the MapReduce result into this HDFS directory."

So, the execution flow is.....

```
Input
/usr/cloudera/weather/input
       ↓
Mapper
weather_mapper.py
        ↓
Shuffle & Sort
        ↓
Reducer
weather_reducer.py
        ↓
Output
/usr/cloudera/weather/output
```

Important note: Hadoop **must not already have the output directory**.If you run the program a second time, delete the previous output first:

```
hdfs dfs -rm -r /usr/cloudera/weather/output
```

6. `-mapper`

`-mapper "python2 weather_mapper.py"`

This tells Hadoop which Mapper program to execute.

Here: `python2`

means use Python 2 to execute: `weather_mapper.py`

So Hadoop effectively runs: `python2 weather_mapper.py`

The Mapper reads input from **STDIN** and produces intermediate key-value pairs through **STDOUT**.

For example, if your weather input contains:

```
2015,01,25.3
2015,02,27.1
2015,03,29.5
```

your Mapper might produce something like:

```
2015    25.3
2015    27.1
2015    29.5
```

7. `-reducer`

`-reducer "python2 weather_reducer.py"`

This tells Hadoop which Reducer program to execute.
`python2`
runs: `weather_reducer.py`
The Reducer receives the Mapper's intermediate output after Hadoop performs **shuffle and sort**.
For example:
```
2015    25.3
2015    27.1
2015    29.5
```
The Reducer can calculate things such as:
```
2015    27.3
```
depending on what your `weather_reducer.py` is programmed to calculate.

---

8. -file weather_mapper.py
`-file weather_mapper.py`
This tells Hadoop:

> "Send/copy my Mapper Python file to the worker nodes where the Map task will execute."

Without making the script available to the Hadoop task, the cluster may not be able to find:
`weather_mapper.py`

---

9. -file weather_reducer.py
`-file weather_reducer.py`
Similarly, this makes: `weather_reducer.py` available to the Hadoop tasks.
So these two: -file `weather_mapper.py` -file `weather_reducer.py`
basically package your Python programs with the Hadoop job.

Complete meaning of entire command:

> **Use Hadoop Streaming to read weather data from HDFS, execute** `weather_mapper.py` **using Python 2, shuffle and sort the Mapper output, execute** `weather_reducer.py` **using Python 2, and store the final result in the HDFS output directory.**
